Links

By Magnus Vaslag Løvøy, Github: https://github.com/magloev/magloevIND320, Streamlit for task2: https://magloevind320task2.streamlit.app/
Will later use main streamlit app after merging the branches

Imports

In [3]:
```
import requests
import pandas as pd
import os
from dotenv import load_dotenv
import xml.etree.ElementTree as ET #Built-in python library used for working with XML
from datetime import datetime, timedelta
import matplotlib.pyplot as plt
from pyspark.sql import functions as F
from pymongo import MongoClient
from pyspark.sql import SparkSession
```

Testing NVE API

This will be used instead of the pd.read in my data_loader.py file

In [4]:
```
url = "https://biapi.nve.no/magasinstatistikk/api/Magasinstatistikk/HentOffentligData"

response = requests.get(url, timeout=60)
response.raise_for_status()

data = response.json()
df = pd.DataFrame(data)

print(df.shape)
```

```
print(df.columns.tolist())
print(df.dtypes)
df.head()
```

```
(14922, 11)
['dato_Id', 'omrType', 'omrnr', 'iso_aar', 'iso_uke', 'fyllingsgrad', 'kapasitet_TWh', 'fylling_TWh', 'neste_Publiseringsdato',
'fyllingsgrad_forrige_uke', 'endring_fyllingsgrad']
dato_Id                        str
omrType                        str
omrnr                        int64
iso_aar                      int64
iso_uke                      int64
fyllingsgrad               float64
kapasitet_TWh              float64
fylling_TWh                float64
neste_Publiseringsdato         str
fyllingsgrad_forrige_uke   float64
endring_fyllingsgrad       float64
dtype: object
```

Out[4]:

	dato_Id	omrType	omrnr	iso_aar	iso_uke	fyllingsgrad	kapasitet_TWh	fylling_TWh	neste_Publiseringsdato	fyllingsgrad_forrige_uke
0	1995-09-03	EL	4	1995	35	0.944721	21.079208	19.913979	0001-01-01T00:00:00	0.936888
1	2020-11-15	EL	1	2020	46	0.974361	6.003264	5.849344	2020-11-25T13:00:00	0.997406
2	2011-08-28	EL	4	2011	34	0.786499	21.079208	16.578772	0001-01-01T00:00:00	0.787193
3	1998-07-05	EL	3	1998	27	0.793969	8.920932	7.082947	0001-01-01T00:00:00	0.733812
4	2011-03-27	EL	4	2011	12	0.300820	21.079208	6.341054	0001-01-01T00:00:00	0.311122

Testing ENTSO-E API

In [5]:
```
load_dotenv()

url = "https://web-api.tp.entsoe.eu/api"

#All the parameters needed to check API connection works, fetched from the ENTSOE API documentation
#Testing the connection first with the area NO2-DK1 from 1st to 2nd October 2026.
params = {
    "securityToken": os.getenv("ENTSOE_API_KEY"),
    "documentType": "A11",
    "out_Domain": "10YNO-2--------T",
    "in_Domain": "10YDK-1--------W",
    "periodStart": "202610010000",
    "periodEnd": "202610020000"
}

response = requests.get(url, params=params, timeout=60)

print("Status:", response.status_code)
print(response.text[:1500])
```

```
Status: 200
<?xml version="1.0" encoding="utf-8"?>
<Publication_MarketDocument xmlns="urn:iec62325.351:tc57wg16:451-3:publicationdocument:7:0">
  <mRID>64bd23636d014265ae3126566cff42f5</mRID>
  <revisionNumber>1</revisionNumber>
  <type>A11</type>
  <sender_MarketParticipant.mRID codingScheme="A01">10X1001A1001A450</sender_MarketParticipant.mRID>
  <sender_MarketParticipant.marketRole.type>A32</sender_MarketParticipant.marketRole.type>
  <receiver_MarketParticipant.mRID codingScheme="A01">10X1001A1001A450</receiver_MarketParticipant.mRID>
  <receiver_MarketParticipant.marketRole.type>A33</receiver_MarketParticipant.marketRole.type>
  <createdDateTime>2026-10-19T09:47:28Z</createdDateTime>
  <period.timeInterval>
    <start>2026-10-01T00:00Z</start>
    <end>2026-10-02T00:00Z</end>
  </period.timeInterval>
    <TimeSeries>
      <mRID>1</mRID>
      <businessType>A66</businessType>
      <in_Domain.mRID codingScheme="A01">10YDK-1--------W</in_Domain.mRID>
      <out_Domain.mRID codingScheme="A01">10YNO-2--------T</out_Domain.mRID>
      <quantity_Measure_Unit.name>MAW</quantity_Measure_Unit.name>
      <curveType>A03</curveType>
        <Period>
          <timeInterval>
            <start>2026-10-01T00:00Z</start>
            <end>2026-10-02T00:00Z</end>
          </timeInterval>
          <resolution>PT15M</resolution>
            <Point>
              <position>1</position>
              <quantity>605.97</quantity>
            </Point>
            <Point>
              <position>2</posi
```

XML Extraction

The NVE API returns JSON format, while the ENTSO-E returns in XML, so we would need to extract the relevant data from XML.

In [6]:
```
#response.content contains the raw XML data we fetched from ENTSO-E API. fromstring converts the raw data into something Pytho
root = ET.fromstring(response.content)

#Creates a dictionary by EXtracting the namespace from the root tag. This creates the alias "ns" which provides the dictionary
# the actual namespace.
ns = {"ns": root.tag.split("}")[0].strip("{")}

#"ns.:TimeSeries" specifies the element we are looking for using the alias ns, and "ns" provides the dictionary which translat
# the alias into the actual namespace.
series = root.find("ns:TimeSeries", ns)

print("Time series found:", series is not None)

"""Parameters used here:
    Start: start time in YYYY-MM-DDTHH:MMZ format
    End: end time in YYYY-MM-DDTHH:MMZ format
    Resolution: time period/cycle of the data: Here it is PT15M, which means quantity is recorded every 15 minutes
    Quantity: Average electricity flow in MW for the given resolution.
"""
if series is not None:
    period = series.find("ns:Period", ns)

    print("Start:", period.findtext("ns:timeInterval/ns:start", namespaces=ns))
    print("End:", period.findtext("ns:timeInHterval/ns:end", namespaces=ns))
    print("Resolution:", period.findtext("ns:resolution", namespaces=ns))

    for point in period.findall("ns:Point", ns)[:5]:
        print(
            "Position:", point.findtext("ns:position", namespaces=ns),
            "Quantity:", point.findtext("ns:quantity", namespaces=ns)
        )
```

```
Time series found: True
Start: 2026-10-01T00:00Z
End: None
Resolution: PT15M
Position: 1 Quantity: 605.97
Position: 2 Quantity: 647.22
Position: 3 Quantity: 653.14
Position: 4 Quantity: 582.93
Position: 5 Quantity: 452.13
```

Extracting XML to Pandas dataframe

Now we are extracting the XML to a pandas dataframe, which we will later put in to cassandra using spark. For now Ive just used a small period of one day, and only NO-2 to DK-1. Next step will be to do the same for the whole two year period.

In [7]:
```
root = ET.fromstring(response.content)
ns = {"ns": root.tag.split("}")[0].strip("{")}

records = []

for series in root.findall("ns:TimeSeries", ns):
    for period in series.findall("ns:Period", ns):

        #Extract start time. timeInterval/ns:start tells python to navigate two levels down from period to find start time.
        #We get a string containing a "Z" which just represents UTC time, so replace it with +00:00 to make it valid string fo
        #datetime.fromisoformat.
        start = datetime.fromisoformat(
            period.findtext(
                "ns:timeInterval/ns:start",
                namespaces=ns
            ).replace("Z", "+00:00")
        )
        #Ensures the resolution is PT15M so that the convertion from MW to MWh is correct.
        resolution = period.findtext("ns:resolution", namespaces=ns)
        if resolution != "PT15M":
            raise ValueError(f"Unexpected resolution: {resolution}")
        interval = timedelta(minutes=15)
```

```
        for point in period.findall("ns:Point", ns):

            position = int(point.findtext("ns:position", namespaces=ns))
            quantity = float(point.findtext("ns:quantity", namespaces=ns))

            timestamp = start + (position - 1) * interval
            energy_MWh = quantity / 4 # Convert MW to MWh for 15-minute intervals

            records.append({
                "timestamp": timestamp,
                "from_area": "NO2",
                "to_area": "DK1",
                "power_MW": quantity,
                "energy_MWh": energy_MWh
            })

df = pd.DataFrame(records)

display(df.head())
print("Number of observations:", len(df))
```

	timestamp	from_area	to_area	power_MW	energy_MWh
0	2026-10-01 00:00:00+00:00	NO2	DK1	605.97	151.4925
1	2026-10-01 00:15:00+00:00	NO2	DK1	647.22	161.8050
2	2026-10-01 00:30:00+00:00	NO2	DK1	653.14	163.2850
3	2026-10-01 00:45:00+00:00	NO2	DK1	582.93	145.7325
4	2026-10-01 01:00:00+00:00	NO2	DK1	452.13	113.0325

```
Number of observations: 96
```

API Retrieving for two years

First combining the code written earlier with some new code for the timestamps, to check if we still can retrieve the data. Decided to split the dataset into 4 API calls, because max length per API call was 1 year. Then we get 4 six month periods.

In [8]:
```
load_dotenv()

url = "https://web-api.tp.entsoe.eu/api"

#All the parameters needed to check API connection works, fetched from the ENTSOE API documentation.
#Here periodStart and periodEnd are left empty and they will be filled in the loop below.
params = {
    "securityToken": os.getenv("ENTSOE_API_KEY"),
    "documentType": "A11",
    "out_Domain": "10YNO-2--------T",
    "in_Domain": "10YDK-1--------W",
    "periodStart": "",
    "periodEnd": ""
}

start_date = pd.Timestamp("2024-10-09", tz="UTC")
end_date = pd.Timestamp("2026-10-09", tz="UTC")

date_ranges = []

current = start_date

while current < end_date:
    #Calculate the next date, which is either 6 months from the current date or the end date, whichever is earlier.
    next_date = min(current + pd.DateOffset(months=6), end_date)

    date_ranges.append((current, next_date))

    current = next_date

responses = []

for start, end in date_ranges:
    #Convert timestamps in to correct format
    params["periodStart"] = start.strftime("%Y%m%d%H%M")
    params["periodEnd"] = end.strftime("%Y%m%d%H%M")

    response = requests.get(url, params=params, timeout=60)
    #Checks for HTTP errors, if this is not included the code will continue to run even if the request fails.
    response.raise_for_status()
```

```
    responses.append(response.content)

    print(f"Retrieved data from {start.date()} to {end.date()}")
```

```
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
```

Connecting it together

Using the same XML to Pandas extraction as earlier, but now looping through the 4 XML documents we have.

In [9]:
```
records = []

for response in responses:
    root = ET.fromstring(response)
    ns = {"ns": root.tag.split("}")[0].strip("{")}

    for series in root.findall("ns:TimeSeries", ns):
        for period in series.findall("ns:Period", ns):

            #Extract start time. timeInterval/ns:start tells python to navigate two levels down from period to find start time
            #We get a string containing a "Z" which just represents UTC time, so replace it with +00:00 to make it valid strin
            #datetime.fromisoformat.
            start = datetime.fromisoformat(
                period.findtext(
                    "ns:timeInterval/ns:start",
                    namespaces=ns
                ).replace("Z", "+00:00")
            )
            #Lookdt at the dataset and saw that the resolution is either PT15M or PT60M, so we need to handle both cases.
            resolution = period.findtext("ns:resolution", namespaces=ns)
            if resolution == "PT15M":
                interval = timedelta(minutes=15)
            elif resolution == "PT60M":
                interval = timedelta(minutes=60)
            else:
```

```
                raise ValueError(f"Unexpected resolution: {resolution}")

            for point in period.findall("ns:Point", ns):

                position = int(point.findtext("ns:position", namespaces=ns))
                quantity = float(point.findtext("ns:quantity", namespaces=ns))

                timestamp = start + (position - 1) * interval
                energy_MWh = quantity * (interval.total_seconds()/3600)  # Convert MW to MWh depending on interval length

                records.append({
                    "timestamp": timestamp,
                    "from_area": "NO2",
                    "to_area": "DK1",
                    "power_MW": quantity,
                    "energy_MWh": energy_MWh,
                    "interval_minutes": int(interval.total_seconds() / 60)
                })

df = pd.DataFrame(records)

df = df.sort_values("timestamp").reset_index(drop=True)

display(df.head())
display(df.tail())

print("Number of observations:", len(df))
print("First timestamp:", df["timestamp"].min())
print("Last timestamp:", df["timestamp"].max())
print("Duplicate timestamps:", df["timestamp"].duplicated().sum())
print(df["interval_minutes"].value_counts())

print("Total hours covered:",
      df["interval_minutes"].sum() / 60)
```

	timestamp	from_area	to_area	power_MW	energy_MWh	interval_minutes
0	2024-10-09 00:00:00+00:00	NO2	DK1	701.10	701.10	60
1	2024-10-09 01:00:00+00:00	NO2	DK1	615.15	615.15	60
2	2024-10-09 02:00:00+00:00	NO2	DK1	698.33	698.33	60
3	2024-10-09 03:00:00+00:00	NO2	DK1	696.65	696.65	60
4	2024-10-09 04:00:00+00:00	NO2	DK1	811.06	811.06	60

	timestamp	from_area	to_area	power_MW	energy_MWh	interval_minutes
40824	2026-10-08 09:00:00+00:00	NO2	DK1	0.94	0.235	15
40825	2026-10-08 09:15:00+00:00	NO2	DK1	2.06	0.515	15
40826	2026-10-08 09:30:00+00:00	NO2	DK1	0.00	0.000	15
40827	2026-10-08 15:45:00+00:00	NO2	DK1	0.06	0.015	15
40828	2026-10-08 16:00:00+00:00	NO2	DK1	0.00	0.000	15

```
Number of observations: 40829
First timestamp: 2024-10-09 00:00:00+00:00
Last timestamp: 2026-10-08 16:00:00+00:00
Duplicate timestamps: 0
interval_minutes
15    36899
60     3930
Name: count, dtype: int64
Total hours covered: 13154.75
```

Bidding-zones

Went through all bidding-zones in the transparency platform (NO1-NO5) to find all the relevant cross-border transfers. This is the list i got:

- NO1 - SE3

- NO2 - DE/LU
- NO2 - DK1
- NO2 - GB
- NO2 - NL
- NO3 - SE2
- NO4 - FI
- NO4 - SE1
- NO4 - SE2

This means 9 different connections, which will be 18 different directions (Both export and import). NO5 only connects to other Norwegian zones. We will use this later to display all of the different zones. For now, we only look at NO2-DK1.

Transfer Data Function

We have now looked at data for two years, but only in the areas from NO2 to DK1. We would want to get the data for the whole of the 5 Norwegian areas, both import and export. Therefore the next step is to make a function which:

- Generates six-month date ranges up to two years,
- Request data from ENTSO-E API for each date
- Parse the XML responses
- Convert these responses into a pd dataframe

In [10]:
```
def get_transfer_data(out_domain, in_domain, start_date, end_date):
    """
    Fetches electricity transfer data from the ENTSOE API for the specified domains and date range.

    Parameters:
    - out_domain (str): Exporting domain
    - in_domain (str): Importing domain.
    - start_date (str): Start date
    - end_date (str): End date

    Returns:
```

```
    - pd dataframe with the Electricity transfer data including timestamps, areas, power (MW) and energy (MWh)
    """

    load_dotenv()

    url = "https://web-api.tp.entsoe.eu/api"
    start_date = pd.Timestamp(start_date, tz="UTC")
    end_date = pd.Timestamp(end_date, tz="UTC")

    #All the parameters needed to check API connection works, fetched from the ENTSOE API documentation.
    params = {
        "securityToken": os.getenv("ENTSOE_API_KEY"),
        "documentType": "A11", #A11 is the document type for electricity transfers between areas.
        "out_Domain": out_domain,
        "in_Domain": in_domain,
        "periodStart" : start_date,
        "periodEnd": end_date
    }

    date_ranges = []

    current = start_date

    while current < end_date:
        #Calculate the next date, which is either 6 months from the current date or the end date, whichever is earlier.
        next_date = min(current + pd.DateOffset(months=6), end_date)

        date_ranges.append((current, next_date))

        current = next_date

    responses = []

    for start, end in date_ranges:

        params["periodStart"] = start.strftime("%Y%m%d%H%M")
        params["periodEnd"] = end.strftime("%Y%m%d%H%M")

        response = requests.get(url, params=params, timeout=60)
        #Checks for HTTP errors, if this is not included the code will continue to run even if the request fails.
        response.raise_for_status()
```

```
        responses.append(response.content)

        print(f"Retrieved data from {start.date()} to {end.date()}")

    records = []

    for response in responses:
        root = ET.fromstring(response)

        #The normal response from the ENTSO-E API is a Publication_MarketDocument, but if there is an error the response will
        if "Acknowledgement_MarketDocument" in root.tag:
            raise ValueError(
                f"ENTSO-E returned an error: "
                f"{ET.tostring(root, encoding='unicode')}" #Converts the error message into readable string.
            )
        ns = {"ns": root.tag.split("}")[0].strip("{")}

        for series in root.findall("ns:TimeSeries", ns):
            for period in series.findall("ns:Period", ns):

                #Extract start time. timeInterval/ns:start tells python to navigate two levels down from period to find start
                #We get a string containing a "Z" which just represents UTC time, so replace it with +00:00 to make it valid s
                #datetime.fromisoformat.
                start = datetime.fromisoformat(
                    period.findtext(
                        "ns:timeInterval/ns:start",
                        namespaces=ns
                    ).replace("Z", "+00:00")
                )

                resolution = period.findtext("ns:resolution", namespaces=ns)
                if resolution == "PT15M":
                    interval = timedelta(minutes=15)
                elif resolution == "PT60M":
                    interval = timedelta(minutes=60)
                else:
                    raise ValueError(f"Unexpected resolution: {resolution}")

                for point in period.findall("ns:Point", ns):
```

```
                    position = int(point.findtext("ns:position", namespaces=ns))
                    quantity = float(point.findtext("ns:quantity", namespaces=ns))

                    timestamp = start + (position - 1) * interval
                    energy_MWh = quantity * (interval.total_seconds()/3600)  # Convert MW to MWh depending on interval length

                    records.append({
                        "timestamp": timestamp,
                        "from_area": out_domain,
                        "to_area": in_domain,
                        "power_mw": quantity,
                        "energy_mwh": energy_MWh,
                        "interval_minutes": int(interval.total_seconds() / 60)
                    })

    df = pd.DataFrame(records)
    return df

#All the relevant zones we found in the API documentation.
zones = {
    "NO1": "10YNO-1--------2",
    "NO2": "10YNO-2--------T",
    "NO3": "10YNO-3--------J",
    "NO4": "10YNO-4--------9",
    "SE1": "10Y1001A1001A44P",
    "SE2": "10Y1001A1001A45N",
    "SE3": "10Y1001A1001A46L",
    "DK1": "10YDK-1--------W",
    "FI": "10YFI-1--------U",
    "NL": "10YNL----------L",
    "GB": "10YGB----------A",
    "DE/LU": "10Y1001A1001A82H"
}

#List containing all the actual connections we want to check.
connections = [
    ("NO1", "SE3"),
    ("NO2", "DE/LU"),
    ("NO2", "DK1"),
    ("NO2", "GB"),
    ("NO2", "NL"),
```

```
    ("NO3", "SE2"),
    ("NO4", "FI"),
    ("NO4", "SE1"),
    ("NO4", "SE2")
]

all_transfers = []

for area_a, area_b in connections:

    print(f"Retrieving {area_a} to {area_b}")

    # Retrieve both directions
    df_export = get_transfer_data(
        zones[area_a],
        zones[area_b],
        "2024-10-09",
        "2026-10-09"
    )

    df_import = get_transfer_data(
        zones[area_b],
        zones[area_a],
        "2024-10-09",
        "2026-10-09"
    )

    # Combine both directions
    all_transfers.extend([df_export, df_import])

# Combine all connections into one DataFrame
df_transfers = pd.concat(
    all_transfers,
    ignore_index=True
)

df_transfers = df_transfers.sort_values(
    "timestamp"
).reset_index(drop=True)
```

```
print("Total observations:", len(df_transfers))
display(df_transfers.head())
```

```
Retrieving NO1 to SE3
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieving NO2 to DE/LU
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieving NO2 to DK1
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieving NO2 to GB
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieving NO2 to NL
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
```

```
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieving NO3 to SE2
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieving NO4 to FI
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieving NO4 to SE1
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieving NO4 to SE2
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Retrieved data from 2024-10-09 to 2025-04-09
Retrieved data from 2025-04-09 to 2025-10-09
Retrieved data from 2025-10-09 to 2026-04-09
Retrieved data from 2026-04-09 to 2026-10-09
Total observations: 479084
```

	timestamp	from_area	to_area	power_mw	energy_mwh	interval_minutes
0	2024-10-09 00:00:00+00:00	10YNO-1--------2	10Y1001A1001A46L	0.0000	0.0000	60
1	2024-10-09 00:00:00+00:00	10Y1001A1001A44P	10YNO-4--------9	466.2325	466.2325	60
2	2024-10-09 00:00:00+00:00	10YNO-4--------9	10Y1001A1001A44P	0.0000	0.0000	60
3	2024-10-09 00:00:00+00:00	10YFI-1--------U	10YNO-4--------9	11.6700	2.9175	15
4	2024-10-09 00:00:00+00:00	10YNO-4--------9	10YFI-1--------U	0.0000	0.0000	15

Checking to ensure that all the connections are included.

In [11]:
```
print("Total observations:", len(df_transfers))

display(
    df_transfers.groupby(["from_area", "to_area"])
    .agg(
        observations=("energy_mwh", "count"),
        total_energy_mwh=("energy_mwh", "sum")
    )
)
```

```
Total observations: 479084
```

from_area	to_area	observations	total_energy_mwh
10Y1001A1001A44P	10YNO-4--------9	15982	1.369166e+06
10Y1001A1001A45N	10YNO-3--------J	31743	3.611485e+06
	10YNO-4--------9	16076	4.303786e+05
10Y1001A1001A46L	10YNO-1--------2	33930	1.204588e+07
10Y1001A1001A82H	10YNO-2--------T	16537	3.041414e+06
10YDK-1--------W	10YNO-2--------T	18469	3.934221e+06
10YFI-1--------U	10YNO-4--------9	17247	8.764041e+04
10YGB----------A	10YNO-2--------T	9156	1.785937e+06
10YNL----------L	10YNO-2--------T	9372	8.051250e+05
10YNO-1--------2	10Y1001A1001A46L	12599	3.080987e+06
10YNO-2--------T	10Y1001A1001A82H	47364	1.134989e+07
	10YDK-1--------W	40829	1.394180e+07
	10YGB----------A	57766	1.738382e+07
	10YNL----------L	28770	3.516234e+06
10YNO-3--------J	10Y1001A1001A45N	14484	1.500159e+06
10YNO-4--------9	10Y1001A1001A44P	30779	2.927222e+06
	10Y1001A1001A45N	29673	9.642895e+05
	10YFI-1--------U	48308	5.537752e+05

Initializing Pyspark-Cassandra connection

Used for storing the dataset to cassandra table with spark

In [15]:
```
spark = (
    SparkSession.builder
    .appName("ENTSOE_Electricity_Transfers")
    .config(
        "spark.jars.packages",
        "com.datastax.spark:spark-cassandra-connector_2.12:3.5.1"
    )
    .config("spark.cassandra.connection.host", "127.0.0.1")
    .config("spark.cassandra.connection.port", "9042")
    .getOrCreate()
)
```

Making sure it works

In [16]:
```
test_df = (
    spark.read
    .format("org.apache.spark.sql.cassandra")
    .options(
        table="transfers",
        keyspace="electricity"
    )
    .load()
)

test_df.printSchema()
```

```
root
 |-- from_area: string (nullable = false)
 |-- to_area: string (nullable = false)
 |-- timestamp: timestamp (nullable = true)
 |-- energy_mwh: double (nullable = true)
 |-- interval_minutes: integer (nullable = true)
 |-- power_mw: double (nullable = true)
```

Real dataframe

In [17]:
```
spark_df = spark.createDataFrame(df_transfers)
spark_df.printSchema()
spark_df.show(5)
spark_df.count()
```

```
root
 |-- timestamp: timestamp (nullable = true)
 |-- from_area: string (nullable = true)
 |-- to_area: string (nullable = true)
 |-- power_mw: double (nullable = true)
 |-- energy_mwh: double (nullable = true)
 |-- interval_minutes: long (nullable = true)

+-------------------+----------------+----------------+--------+----------+----------------+
|          timestamp|       from_area|         to_area|power_mw|energy_mwh|interval_minutes|
+-------------------+----------------+----------------+--------+----------+----------------+
|2024-10-09 02:00:00|10YNO-1--------2|10Y1001A1001A46L|     0.0|       0.0|              60|
|2024-10-09 02:00:00|10Y1001A1001A44P|10YNO-4--------9|466.2325|  466.2325|              60|
|2024-10-09 02:00:00|10YNO-4--------9|10Y1001A1001A44P|     0.0|       0.0|              60|
|2024-10-09 02:00:00|10YFI-1--------U|10YNO-4--------9|   11.67|    2.9175|              15|
|2024-10-09 02:00:00|10YNO-4--------9|10YFI-1--------U|     0.0|       0.0|              15|
+-------------------+----------------+----------------+--------+----------+----------------+
only showing top 5 rows
```

Out[17]: 479084

Writing the data to Cassandra

In [18]:
```
(
    spark_df.write
    .format("org.apache.spark.sql.cassandra")
    .mode("append")
    .options(
        keyspace="electricity",
```

```
        table="transfers"
    )
    .save()
)
```

In [19]:

```
cassandra_df = (
    spark.read
    .format("org.apache.spark.sql.cassandra")
    .options(
        keyspace="electricity",
        table="transfers"
    )
    .load()
)

print("Rows in Cassandra:", cassandra_df.count())
cassandra_df.show(5, truncate=False)
cassandra_df.groupBy("from_area", "to_area").count().show(20, truncate=False)
```

```
Rows in Cassandra: 479034
+----------------+----------------+-------------------+----------+----------------+--------+
|from_area       |to_area         |timestamp          |energy_mwh|interval_minutes|power_mw|
+----------------+----------------+-------------------+----------+----------------+--------+
|10YNO-1--------2|10Y1001A1001A46L|2024-10-09 02:00:00|0.0       |60              |0.0     |
|10YNO-1--------2|10Y1001A1001A46L|2024-10-10 07:00:00|148.84    |60              |148.84  |
|10YNO-1--------2|10Y1001A1001A46L|2024-10-10 08:00:00|79.67     |60              |79.67   |
|10YNO-1--------2|10Y1001A1001A46L|2024-10-10 09:00:00|83.21     |60              |83.21   |
|10YNO-1--------2|10Y1001A1001A46L|2024-10-10 10:00:00|74.94     |60              |74.94   |
+----------------+----------------+-------------------+----------+----------------+--------+
only showing top 5 rows

+----------------+----------------+-----+
|from_area       |to_area         |count|
+----------------+----------------+-----+
|10YNO-3--------J|10Y1001A1001A45N|14484|
|10YNO-2--------T|10YGB----------A|57762|
|10YNO-1--------2|10Y1001A1001A46L|12599|
|10Y1001A1001A82H|10YNO-2--------T|16534|
|10YFI-1--------U|10YNO-4--------9|17243|
|10YNO-2--------T|10YDK-1--------W|40827|
|10Y1001A1001A46L|10YNO-1--------2|33925|
|10Y1001A1001A45N|10YNO-4--------9|16071|
|10YNO-2--------T|10Y1001A1001A82H|47363|
|10YNO-4--------9|10Y1001A1001A44P|30779|
|10YNO-4--------9|10YFI-1--------U|48306|
|10YNO-4--------9|10Y1001A1001A45N|29673|
|10YDK-1--------W|10YNO-2--------T|18465|
|10Y1001A1001A45N|10YNO-3--------J|31738|
|10Y1001A1001A44P|10YNO-4--------9|15977|
|10YNO-2--------T|10YNL----------L|28764|
|10YGB----------A|10YNO-2--------T|9152 |
|10YNL----------L|10YNO-2--------T|9372 |
+----------------+----------------+-----+
```

Making sure the DataFrames matches and that the extraction worked

In [20]:
```
spark_df.orderBy(
    "timestamp", "from_area", "to_area"
```

```
).show(10, truncate=False)

cassandra_df.orderBy(
    "timestamp", "from_area", "to_area"
).show(10, truncate=False)
```

```
+-------------------+----------------+----------------+--------+----------+----------------+
|timestamp          |from_area       |to_area         |power_mw|energy_mwh|interval_minutes|
+-------------------+----------------+----------------+--------+----------+----------------+
|2024-10-09 02:00:00|10Y1001A1001A44P|10YNO-4--------9|466.2325|466.2325  |60              |
|2024-10-09 02:00:00|10Y1001A1001A45N|10YNO-3--------J|431.0725|431.0725  |60              |
|2024-10-09 02:00:00|10Y1001A1001A45N|10YNO-4--------9|139.005 |139.005   |60              |
|2024-10-09 02:00:00|10Y1001A1001A46L|10YNO-1--------2|627.405 |627.405   |60              |
|2024-10-09 02:00:00|10Y1001A1001A82H|10YNO-2--------T|0.0     |0.0       |15              |
|2024-10-09 02:00:00|10YDK-1--------W|10YNO-2--------T|0.0     |0.0       |60              |
|2024-10-09 02:00:00|10YFI-1--------U|10YNO-4--------9|11.67   |2.9175    |15              |
|2024-10-09 02:00:00|10YGB----------A|10YNO-2--------T|0.0     |0.0       |15              |
|2024-10-09 02:00:00|10YNL----------L|10YNO-2--------T|0.0     |0.0       |15              |
|2024-10-09 02:00:00|10YNO-1--------2|10Y1001A1001A46L|0.0     |0.0       |60              |
+-------------------+----------------+----------------+--------+----------+----------------+
only showing top 10 rows

+----------------+----------------+-------------------+----------+----------------+--------+
|from_area       |to_area         |timestamp          |energy_mwh|interval_minutes|power_mw|
+----------------+----------------+-------------------+----------+----------------+--------+
|10Y1001A1001A44P|10YNO-4--------9|2024-10-09 02:00:00|466.2325  |60              |466.2325|
|10Y1001A1001A45N|10YNO-3--------J|2024-10-09 02:00:00|431.0725  |60              |431.0725|
|10Y1001A1001A45N|10YNO-4--------9|2024-10-09 02:00:00|139.005   |60              |139.005 |
|10Y1001A1001A46L|10YNO-1--------2|2024-10-09 02:00:00|627.405   |60              |627.405 |
|10Y1001A1001A82H|10YNO-2--------T|2024-10-09 02:00:00|0.0       |15              |0.0     |
|10YDK-1--------W|10YNO-2--------T|2024-10-09 02:00:00|0.0       |60              |0.0     |
|10YFI-1--------U|10YNO-4--------9|2024-10-09 02:00:00|2.9175    |15              |11.67   |
|10YGB----------A|10YNO-2--------T|2024-10-09 02:00:00|0.0       |15              |0.0     |
|10YNL----------L|10YNO-2--------T|2024-10-09 02:00:00|0.0       |15              |0.0     |
|10YNO-1--------2|10Y1001A1001A46L|2024-10-09 02:00:00|0.0       |60              |0.0     |
+----------------+----------------+-------------------+----------+----------------+--------+
only showing top 10 rows
```

Extract from Cassandra to Spark

Now that we have stored the dataset in cassandra, we would like to use it to plot the pie chart and line chart. For this we need following columns:

- energy_mwh
- from_area
- to_area
- timestamp

Pie Chart and Line Chart

Using "F" which is pyspark built-in sql functions to analyze and work with the spark dataframe

In [27]:
```
# Extract relevant columns from Cassandra
extracted_df = cassandra_df.select(
    "timestamp",
    "from_area",
    "to_area",
    "energy_mwh"
)

# Filter for NO2-DK1 in both directions
connection_df = extracted_df.filter(
    (
        (F.col("from_area") == zones["NO2"]) &
        (F.col("to_area") == zones["DK1"])
    ) |
    (
        (F.col("from_area") == zones["DK1"]) &
        (F.col("to_area") == zones["NO2"])
    )
)

# Dataframe for pie chart - year 2025
year_df = connection_df.filter(
    F.year("timestamp") == 2025
```

```
)

# Dataframe for line plot - January 2025
january_df = connection_df.filter(
    (F.col("timestamp") >= "2025-01-01") &
    (F.col("timestamp") < "2025-02-01")
)

# Group observations by hour and direction
hourly_df = (
    january_df
    .groupBy(
        F.date_trunc("hour", "timestamp").alias("hour"),
        "from_area",
        "to_area"
    )
    .agg(F.sum("energy_mwh").alias("energy_mwh"))
    .orderBy("hour")
)

hourly_df.show(5)

# Aggregate total energy for each direction
pie_df = (
    year_df
    .groupBy("from_area", "to_area")
    .agg(F.sum("energy_mwh").alias("total_energy_mwh"))
)

# Convert Spark DataFrames to Pandas for plotting
pie_pd = pie_df.toPandas()
hourly_pd = hourly_df.toPandas()

# Create readable direction labels
hourly_pd["direction"] = hourly_pd["from_area"].apply(
    lambda x: "Export (NO2 to DK1)" if x == zones["NO2"]
    else "Import (DK1 to NO2)"
)

labels = [
    "Export (NO2 to DK1)" if area == zones["NO2"]
```

```
    else "Import (DK1 to NO2)"
    for area in pie_pd["from_area"]
]

# Create figure with two subplots
fig, (ax1, ax2) = plt.subplots(
    1, 2, figsize=(16, 6),
    gridspec_kw={"width_ratios": [1, 2]}
)

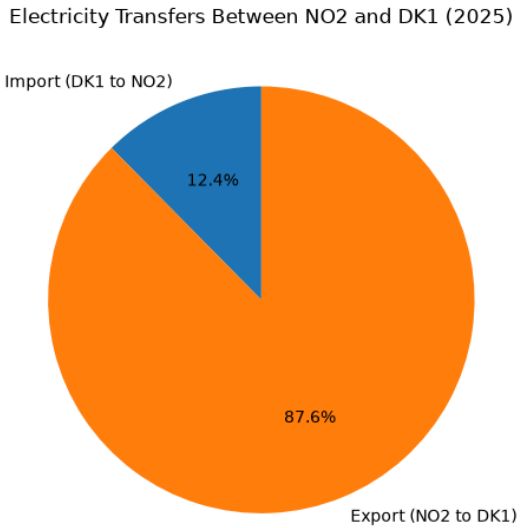
# Pie chart
ax1.pie(
    pie_pd["total_energy_mwh"],
    labels=labels,
    autopct="%1.1f%%",
    startangle=90
)
ax1.set_title("Electricity Transfers Between NO2 and DK1 (2025)")

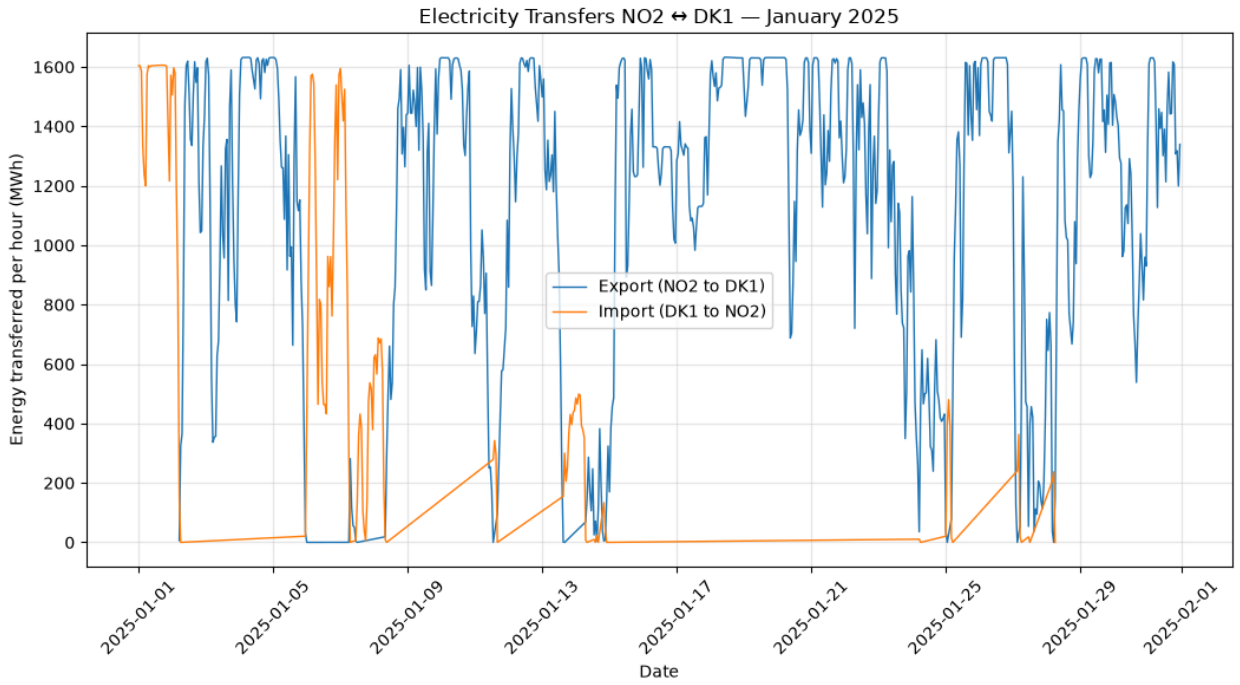
# Line plot
for direction, group in hourly_pd.groupby("direction"):
    ax2.plot(
        group["hour"],
        group["energy_mwh"],
        label=direction,
        linewidth=1
    )

ax2.set_title("Electricity Transfers NO2 ↔ DK1 — January 2025")
ax2.set_xlabel("Date")
ax2.set_ylabel("Energy transferred per hour (MWh)")
ax2.legend()
ax2.grid(alpha=0.3)
ax2.tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

```
+------------------+---------------+---------------+----------+
|              hour|      from_area|        to_area|energy_mwh|
+------------------+---------------+---------------+----------+
|2025-01-01 00:00:00|10YDK-1--------W|10YNO-2--------T| 1604.1425|
|2025-01-01 01:00:00|10YDK-1--------W|10YNO-2--------T|   1603.96|
|2025-01-01 02:00:00|10YDK-1--------W|10YNO-2--------T|   1585.13|
|2025-01-01 03:00:00|10YDK-1--------W|10YNO-2--------T| 1330.3925|
|2025-01-01 04:00:00|10YDK-1--------W|10YNO-2--------T| 1237.3875|
+------------------+---------------+---------------+----------+
only showing top 5 rows
```





Connection with MongoDB

Connection to MongoDB Atlas

In [28]:
```
from pymongo import MongoClient
from dotenv import load_dotenv
```

```
import os

# Load environment variables
load_dotenv()

# Connect to MongoDB Atlas
atlas_client = MongoClient(os.getenv("MONGO_URI"))

# Verify connection
atlas_client.admin.command("ping")
print("Connected to MongoDB Atlas")
```

```
Connected to MongoDB Atlas
```

In [ ]:

```
# Make sure we're using MongoDB Atlas
collection = atlas_client["ind320"]["transfers"]

total_inserted = 0

for area_a, area_b in connections:

    print(f"Processing {area_a} and {area_b}")

    # Filter to get both directions. We use "&" as AND inside the clause to only retrieve rows for the two connections.
    # We use "|" as OR outside to include either direction.
    connection_df = extracted_df.filter(
        (
            (F.col("from_area") == zones[area_a]) &
            (F.col("to_area") == zones[area_b])
        ) |
        (
            (F.col("from_area") == zones[area_b]) &
            (F.col("to_area") == zones[area_a])
        )
    )

    # Convert this connection to Pandas
    connection_pd = connection_df.toPandas()

    # Convert to MongoDB documents
    records = connection_pd.to_dict("records")
```

```
    # Delete existing documents as we will probably run the notebook multiple times so we dont duplicate.
    collection.delete_many({
        "$or": [
            {
                "from_area": zones[area_a],
                "to_area": zones[area_b]
            },
            {
                "from_area": zones[area_b],
                "to_area": zones[area_a]
            }
        ]
    })

    # Insert documents
    if records:
        collection.insert_many(records)

    total_inserted += len(records)

    print(f"Inserted {len(records)} documents")

print(f"Total inserted: {total_inserted}")
```

```
Processing NO1 and SE3
Inserted 46524 documents
Processing NO2 and DE/LU
Inserted 63897 documents
Processing NO2 and DK1
Inserted 59292 documents
Processing NO2 and GB
Inserted 66914 documents
Processing NO2 and NL
Inserted 38136 documents
Processing NO3 and SE2
Inserted 46222 documents
Processing NO4 and FI
Inserted 65549 documents
Processing NO4 and SE1
Inserted 46756 documents
Processing NO4 and SE2
Inserted 45744 documents
Total inserted: 479034
```

In [31]:
```
atlas_client.admin.command("ping")
```

Out[31]:
```
{'ok': 1.0}
```

AI Usage

I used ChatGPT throughout the project to support me with code implementation, troubleshooting and understanding the pipeline and the different tools used.

Initially I used AI to break down the project to smaller pieces and understand how the ENTSOE API worked and what it looked like. I spent a lot of time back and forth trying to understand the XML structure and how to extract it to Pandas. I mostly used AI in the start to investigate the data by giving me code snippets to investigate the missing data. For Spark, Cassandra and MongoDB it was very useful to set up the connections and troubleshoot inbetween. I used a lot of time trying to understand the logic and purpose behind it while AI gave much of the code. Overall I focused less on writing code manually and more on understanding the data set and the pipeline of XML -> Spark -> Cassandra -> MongoDB. I tested the implementations and made decisions about the dataset, visualization and structure throughout the project.

Log

First I connected to the NVE API, pretty straightforward and all formats and everything was already the same, so it was pretty much only removing the reading from csv and changing it with the actual API connection.

Started with exploring the ENTSO-E Transparency Platform and the API documentaiton. First i only focused on the NO2-DK1 connection to get an understanding of the dataset before expanding to the other connections. I spent some time understanding the XML structure the API retrieves. The main elements are TimeSeries (NO2-DK1 for example), Period (Defines the time interval and resolution) and Point (Contains position and quantity(MW)). The dataset contains two resolutions, PT15M (15 minute period) and PT60M. Since the quantity from the XML is measured in MW, we need to multiply by the interval duration in hours to get MWH. The API requests had an upper limit of 1 year per retrieval. I decided to retrieve the two-year period in 4 API retrievals of 6 months each just to be safe.

I spent quite some time investigating the transfer data and missing values. Initially found out that only around 13000 hours of around 17000 hours were accounted for in the dataset. But after comparing both directions in each connection I found out that over 99% of the expected time was covered by atleast one direction. This brings us to netted and non-netted reports. Some of the time they report netted values, which means in many cases, in NO2-DK1 atleast, the flow NO2 to DK1 was very high, while flow DK1-NO2 was 0. However some intervals contained positive values in both directions suggesting the reporting is more complex. The dataset still provided sufficient coverage so i decided not to temper with the dataset with imputing or similar.

Converting to Spark DataFrame after exploring in Pandas. Set up Cassandra through docker, connected to spark and inserted the dataset into cassandra. Afterwards used spark again to extract the relevant data for plotting. Then we set up MongoDB Atlas to store the retrieved observations, and expanded the connections to show all 9 connections.

For Streamlit it was pretty straightforward just using what I already wrote in the notebook, with small changes using streamlit functions instead. Created a separate deployment for task 2 Streamlit app, which will be removed after merging the branches, after the peer-review and feedback.